Summer Linear Attention

デルタ状態と有界KV検索を組み合わせたメモリの参照実装研究

summer

2026年10月24日

技術論文草稿 / 未査読

研究論文草稿。 本稿は、学習済みモデルではないNumPy参照実装の仕様と性質を記述する。新たなベンチマーク測定結果は報告しない。第6節は未実施の評価計画である。対象コードはSummerLinearAttention v0.1.0、コミット036512212ed0に固定する。

概要

固定サイズの再帰的メモリと明示的なキー・バリュー（KV）ストアは、系列中の対応関係を異なる方法で保持する。デルタ則に基づく状態はコンパクトな線形予測器を構成し、有界アーカイブは一部の観測値を状態へ圧縮せずに保存する。本稿では、アーカイブのキーとnull候補に対する単一のsoftmaxを用いて両者を組み合わせる、小規模な参照実装Summer Linear Attention（SLA）を記述する。null候補の重みは、保存された値ではなく状態予測に割り当てる。実装は、正規化したキーとクエリ、固定の更新パラメータ、挿入順FIFOアーカイブ、および全候補を走査する検索からなる。コードに一致する仕様を示し、更新と読出しの基本的性質を導出するとともに、有限のsoftmax鋭さ、キーの衝突、アーカイブ占有数、除去による失敗条件を明らかにする。任意に利用できる共有状態の残差読出しは、既定手法と区別して解析する。次元と容量を固定した場合、常駐配列の容量は有界であり、系列全体の処理量は系列長に対して線形となる。ただし、これはGPU効率や、増大する文脈からの厳密な検索を意味しない。再帰状態とキャッシュの併用、およびsentinelによるフォールバック混合には先行研究が存在する。本稿の位置付けは、透明な参照実装の研究と反証可能な評価計画の提示であり、構造上の先行性や実証済みの性能優位性の主張ではない。

キーワード： 線形注意、デルタ則、連想メモリ、有界KVキャッシュ、sentinel混合、参照実装。

1. はじめに

系列モデルには、何を保持し、どのように取り出すかという設計上の選択がある。カーネルに基づく線形注意は、過去のすべてのKV対を保持しない再帰的な計算として表現できる[1]。デルタ則による高速重みメモリは、単純な加算型書込みの代わりに、その時点のキー・バリュー予測に対する補正を書き込む[2]。その後、系列方向の並列化やゲート付きの変種により、学習済み言語モデルへの展開が進んだ[3, 4]。ただし、コンパクトな状態が任意の対応関係を相互に独立して保持できるわけではない。相関したキー方向への更新は干渉を生じさせる。

有界の明示的アーカイブは、状態とは異なる表現手段となる。エントリが残っている間は、状態予測が変化しても保存値自体を保持できる。一方で、クエリに対して状態予測と保存値のどちらを利用すべきかという選択が新たに必要になる。SLAは、これらの選択を小規模な実行可能仕様として切り出す。アーカイブの管理を単純化し、学習された射影、制御器、エンドツーエンドの学習系を前提とせずに読出しを検討できるようにする。

動機となる仮説は条件付きである。値がキーから予測できる成分と個別項目に固有の偏差を含むなら、状態が構造の一部を要約し、アーカイブが一部の例外を保持できる可能性がある。しかし、この

相補性を提案する混合則が実際に利用できるかどうかは、アクセス分布、容量、キーの幾何、および較正に依存する。値がキーと独立な場合や、近いキーが矛盾する値を持つ場合には、利点が乏しいか、かえって誤差を増やす可能性がある。本稿では、この動機を実験によって確認された知見として扱わない。

本稿の内容は三つに整理できる。第一に、公開参照実装の書込み、同一IDの上書き、除去、読出しの厳密な意味を規定する。第二に、元の値を保存することと、その値を厳密に出力することを区別し、実装に直結する性質と反例を導出する。第三に、読出し、保持方針、およびメモリ予算を切り分けた評価計画を示す。再帰状態とキャッシュの組合せや、softmaxの重みをフォールバック予測器へ割り当てる原理について、初出の主張は行わない。

2. 関連研究と対象範囲

再帰的注意とデルタ則。 Katharopoulosら[1]は、カーネル特徴と自己回帰的な漸化式によって線形注意を定式化した。Schlagら[2]は線形Transformerと高速重みプログラミングの関係を示し、デルタ則の更新を検討した。Yangら[3]はその系列方向の並列計算を扱い、Gated Delta Networks[4]は減衰と方向を限定した補正を組み合わせる。SLAはスカラー減衰を伴うデルタ漸化式を使用するが、これらのモデル全体や最適化された学習アルゴリズムを実装してはいない。

sentinelによるフォールバック混合。 Pointer Sentinel Mixture Models[5]は、ポインタのスコアとsentinelのスコアを同時に正規化し、sentinelの重みを語彙予測器に割り当てる。SLAは類似の構成を連続値に適用し、固定スカラーのnull logitに対応する重みをデルタ状態の予測へ割り当てる。したがって、フォールバック混合の原理には先行研究がある。異なるのは出力空間とパラメータ化であり、softmax内にフォールバックを設ける一般原理ではない。

再帰状態と有界キャッシュ。 HOLA[6]は特に近い先行研究であり、デルタ状態、有界の元KVキャッシュ、null sinkを組み合わせる。残差に基づく保持、鋭いキャッシュ検索、および加算型のゲート付き統合を用いる。これに対しSLAはFIFO保持を採用し、共同正規化された混合のnull重みを状態予測に割り当てる。この違いは本稿の検討対象を特定するものであり、優位性を示すものではない。Learnable Token Eviction[7]も線形注意と疎な注意のハイブリッドにおける有界KV保持を扱うが、SLAには学習可能な除去モジュールはない。

関連プロジェクトSDA。 Summer Delta Attention（SDA）は別の関連プロジェクトである[9]。SLAの文書では、nullを含むソフトな混合と、SDAの信頼度ゲートを伴うハードなtop-1想起を区別している。後述する任意の残差拡張もソフトな混合であり、そのハード選択則の実装ではない。SDAの結果、メモリ量、図をSLAの結果として転用してはならない。

対象は、固定したSLAコミット[8]に含まれる公開クラス`SLAMemory`である。リポジトリは著者をsummerと記載し、コードをMITライセンスで公開している。学習済みモデル、学習するキー・クエリ射影、自動微分を用いた学習系、検索インデックス、最適化されたGPUカーネルは含まれない。本稿は手法と参照実装の研究であり、ベンチマーク論文でもハイブリッド注意の網羅的な総説でもない。

3. 手法

3.1. 構成要素と表記

正の整数d_k, d_vをキーと値の次元、非負整数Bをアーカイブ容量とする。メモリは、ゼロで初期化する行列$S \in \mathbb{R}^{d_k \times d_v}$と、順序付きアーカイブ

$$\mathcal{A} = [(\kappa_i, u_i, \iota_i)]_{i=1}^{m}, \qquad 0 \le m \le B$$

からなる。κ_iは単位ノルムのキー、u_iは保存値、ι_iは書込み時だけ用いる任意の整数IDである。読出しにはIDを渡さない。有限かつ非ゼロの入力キーとクエリは、ユークリッドノルムが1になるように正規化する。値は指定した浮動小数点型に変換した後でコピーする。したがって「元KV」とは、正規化したキーと型変換後の保存値を意味し、正規化前の入力キーや任意精度のデータを意味しない。

パラメータはスカラー減衰$\alpha \in [0, 1]$、書込み強度$\beta \in (0, 1]$、スコア倍率$s > 0$、有限のnull logit cである。既定値は$B = 32$、$\alpha = 1$、$\beta = 0.05$、$s = 32$、$c = 0$、float64であり、float32も使用できる。いずれも固定の設定値であり、学習されたゲートではない。

3.2. デルタ状態への書込み

正規化したキーk_tと値v_tに対する更新は

$$\bar{S}_t = \alpha S_{t-1}, \qquad e_t = v_t - \bar{S}_t^{\top} k_t, \qquad S_t = \bar{S}_t + \beta k_t e_t^{\top} \tag{1}$$

である。予測残差は減衰後の状態から計算する。妥当な書込みは、同じIDへの更新や$B = 0$の場合も含め、必ず状態を更新する。アーカイブの上書きは、過去の書込みが状態へ与えた寄与を取り消さない。APIでは書込みと読出しが独立しており、読出しはその時点までに完了したすべての書込みを見る。系列評価では現在項目の書込み前後のどちらで評価するかを明示し、予測すべき未提示の答えを先に書き込まないようにする必要がある。

3.3. アーカイブへの書込みと除去

状態更新後のアーカイブ処理は、次の順に行う。

1. $B = 0$ならアーカイブを空のままにする。
2. 指定された整数IDに一致する保持エントリがあれば、そのスロットのキーと値を置換する。FIFO内の位置は変えない。
3. 一致がなく空きがあれば、新しいエントリを末尾に追加する。
4. 満杯なら、保持中で最も早く挿入されたエントリを取り除き、残りを前へ詰め、新しいエントリを末尾に追加する。

未指定のIDは既存エントリと一致しない。異なるIDが同一の正規化キーを共有することは許される。この方針は挿入スロット順のFIFOであり、LRUではない。また、同じ場所での上書きが発生する場合、直近の書込み操作を保持するスライディングウィンドウとも異なる。驚きに基づく閾値、効用順位、学習された受入ゲート、クエリを契機としたアーカイブ書込みは存在しない。

3.4. nullを含む読出し

単位クエリqに対して、まず状態予測$f(q) = S^\top q$を計算する。アーカイブが空でなければ、

$$\ell_0 = c, \qquad \ell_i = s\kappa_i^\top q, \qquad w_i = \frac{\exp(\ell_i)}{\sum_{j=0}^{m} \exp(\ell_j)} \tag{2}$$

とし、既定のSLA読出しを

$$y_0(q) = w_0 f(q) + \sum_{i=1}^{m} w_i u_i \tag{3}$$

で定める。実装では指数関数の計算前に最大logitを差し引く。空のアーカイブに対しては$f(q)$を直接返し、null重みを1とする。null候補は状態予測への経路であり、出力ゼロのトークンや「答えが存在しない」という明示的な判定ではない。その重みは正解確率や欠落確率として較正されていない。

3.5. 任意の共有状態・残差読出し

APIは読出し時に指定する固定係数$\lambda \in [0, 1]$も提供する。

$$y_\lambda(q) = y_0(q) + \lambda \sum_{i=1}^{m} w_i S^\top (q - \kappa_i). \tag{4}$$

過去のアーカイブキーに対する予測も含め、すべての項で現在の状態を用いる。式(4)は

$$y_\lambda(q) = \{w_0 + \lambda(1 - w_0)\} f(q) + \sum_{i=1}^{m} w_i \{u_i - \lambda S^\top \kappa_i\} \tag{5}$$

と書き換えられる。$\lambda = 0$は通常のSLAである。$\lambda = 1$では$f(q) + \sum_i w_i (u_i - S^\top \kappa_i)$となり、状態予測にソフトな残差補正を加える。この出力は、$f(q)$と保存値の凸包内にあるとは限らない。単一エントリの選択も厳密な想起も保証しない。λの適応的な選択は公開参照実装の範囲外である。

3.6. 実行可能仕様

アルゴリズム1：SLAの書込みと読出し。 次の擬似コードは、入力検査と型変換を除いて参照実装に対応する。

```
WRITE(key, value, optional id):
    k = unit(key); v = copy(value)
    S = alpha * S
    S = S + beta * outer(k, v - transpose(S) * k)
    if B == 0: return
    if id is supplied and retained:
        replace that slot's key and value; return
    if archive is full: remove first slot
    append (k, v, id)

READ(query, lambda = 0):
    q = unit(query); f = transpose(S) * q
    if archive is empty: return f
    w = softmax([c, s * dot(k_i, q) for each slot i])
    y = w[0] * f + sum_i w[i+1] * v_i
    y = y + lambda * sum_i w[i+1] * transpose(S) * (q-k_i)
    return y
```

4. 解析的性質と失敗条件

本節の主張は、厳密な実数演算に関するものである。式(1)〜(4)から得られる数学的帰結であり、測定した性能や数値安定性の保証ではない。

4.1. 書込みはそのキーにおける残差を縮小する

命題1。 キーが単位ノルムなら、書込み後の残差は

$$v_t - S_t^\top k_t = (1-\beta)e_t \tag{6}$$

を満たす。

証明。 式(1)にk_tを掛け、$k_t^\top k_t = 1$を用いる。更新後の予測は$\bar{S}_t^\top k_t + \beta e_t$なので、式(6)が得られる。したがって$\beta = 1$なら、書込み直後の状態は現在の対を厳密に再現する。ただし、以前のすべての対を保持するわけではない。別の単位キーrに対する減衰後の予測は、書込みによって$\beta(k_t^\top r)e_t$だけ変化する。これにより、キーの相関に依存する干渉が明示される。

4.2. 条件付きの状態ノルム上界

更新は

$$S_t = \alpha(I - \beta k_t k_t^\top)S_{t-1} + \beta k_t v_t^\top \tag{7}$$

と書ける。許容範囲のβに対して、行列$I - \beta k_t k_t^\top$のスペクトルノルムは1以下である。$\|v_t\|_2 \le V$かつ$S_0 = 0$なら、フロベニウスノルムについて

$$\|S_t\|_F \le \begin{cases} \beta V(1-\alpha^t)/(1-\alpha), & 0 \le \alpha < 1, \\ t\beta V, & \alpha = 1 \end{cases} \tag{8}$$

が成り立つ。三角不等式から$\|S_t\|_F \le \alpha\|S_{t-1}\|_F + \beta V$を得て、反復すればよい。第2行は有限時間の上界にすぎず、既定値$\alpha = 1$で時間に一様な有界性を示したものではない。キーの正規化だけでは、任意の入力値の大きさを制限できず、浮動小数点のオーバーフローも防げない。

4.3. 検索はマージン、nullとの競合、値の差に依存する

$\lambda = 0$では、式(3)は現在の状態予測と保存値の凸結合である。目標となる保存値u_jに対して、

$$\|y_0 - u_j\|_2 \le w_0\|f - u_j\|_2 + \sum_{i \ne j} w_i\|u_i - u_j\|_2 \tag{9}$$

が成り立つ。目標キーのコサイン類似度が、ほかのすべての保存キーより少なくとも$\delta > 0$だけ大きいとすると、

$$w_j \ge \left[1 + e^{c - s\kappa_j^\top q} + (m-1)e^{-s\delta}\right]^{-1} \tag{10}$$

を得る。softmaxの分母を$\exp(s\kappa_j^\top q)$で割り、競合項の比をそれぞれ$e^{-s\delta}$で抑えれば証明できる。この下界は、重みを集中させる条件として、キーのマージン、競合数、null logitに対する優位を切り分ける。それらが満たされても、式(9)の誤差は競合する値の大きさと状態誤差に依存する。正しいキーがtop-1であることだけでは不十分である。

4.4. 元の値の保存は厳密な読出しを意味しない

ゼロ初期化状態に対して$v \ne 0$の1対(k, v)を書き込み、その対だけをアーカイブに保存し、$q = k$で問い合わせる場合を考える。式(1)より$f = \beta v$である。$q - k = 0$なので、許容されるどのλでも残差補正は消え、

$$y_\lambda = [1 - w_0(1 - \beta)]v \tag{11}$$

となる。有限logitでは、厳密な実数演算上$w_0 > 0$である。したがって$\beta < 1$なら、キーが完全に一致し、値がそのまま保存されていても、出力は厳密に一致しない。これは数式上の反例であり、測定したベンチマーク結果ではない。

矛盾する近傍でもソフトな平均化が問題になる。保存値がuと$-u$、クエリとの類似度がそれぞれ1と$1-\delta$の場合、アーカイブからの合計寄与は$(w_+ - w_-)u$である。すべての$\delta > 0$において正の値を持つキーが唯一のtop-1であるにもかかわらず、$\delta \to 0$でこの寄与はゼロに近づく。キーが同一なら、読出しがIDを受け取らないため、IDによって曖昧性を解消することもできない。

4.5. アーカイブ占有数はフォールバック動作を変える

m個のエントリがすべて同じスコアℓを持つと、

$$w_0 = \frac{1}{1 + m\exp(\ell - c)} \tag{12}$$

となる。同じスコアの候補を追加するだけでも、個々の一致度は改善しないまま状態の重みが減る。ある容量や占有数で較正したnull logitが、別の条件へそのまま移るとは限らない。これは経路選択のパラメータであり、欠落の証明ではない。参照実装には占有数を補正する仕組みはない。

最後に、式(4)から

$$\|y_\lambda - y_0\|_2 \le 2\lambda(1 - w_0)\|S\|_2 \tag{13}$$

という摂動の上界を得る。クエリとキーがともに単位ノルムであることを用いればよい。この上界は、補正が有益かどうかを示さない。残差混合を既定の読出しより優れていると仮定せず、別に評価すべき理由を示している。

5. 計算量と実装上のメモリ計測

1スカラーがbバイトの型を用いる場合、事前確保された状態、キー、値の配列容量は厳密に

$$M_{\text{arrays}} = b\{d_k d_v + B(d_k + d_v)\} \tag{14}$$

である。`allocated_array_bytes`が測るのはこの量であり、Pythonオブジェクト、ID、アロケータの管理領域、一時配列、周辺プログラムは含まない。また、アーカイブが部分的にしか埋まっていなくても全確保容量を数える。これはプロセス全体のメモリ、ピーク常駐メモリ、言語モデル全体のキャッシュ容量ではない。

状態更新は$O(d_k d_v)$、全走査による読出しは最悪時に$O(d_k d_v + B(d_k + d_v))$の計算量を要する。実装では残差補正に集約したキーと行列積を用いるため、B回の独立な状態射影は行わず、この漸近計算量を保つ。満杯時の書込みでは配列の移動に$O(B(d_k + d_v))$、ID探索に$O(B)$を要することがある。リングバッファは移動量を減らせるが、公開実装はその方式ではない。

各ステップで書込みと読出しを1回ずつ行い、B, d_k, d_vを固定すれば、Nステップ全体の計算量は$O(N)$となり、常駐配列容量はNに依存しない。一方、BをNに比例して増やすと、反復する全走査によって総計算量が$O(N^2)$となり得る。本稿で「線形注意」と呼ぶ際の線形性は、この固定容量における系列長方向の計算量を指す。読出し全体がクエリに対して線形という意味ではなく、このNumPy実装が特定の注意カーネルより高速だとも主張しない。

通常の書込み変更に先立ち入力検査を行い、ゼロや非有限のキー・クエリ、不正な値の形状を拒否する。キーの正規化では先に絶対値の最大成分で尺度を調整し、ノルム計算時のオーバーフローの危険を抑える。ただし、これらは網羅的な数値検証ではない。有限の値を入力しても、状態の中間演算が非有限になる可能性は残る。機能テストは含まれるが、それだけで精度、処理速度、学習済みモデルの挙動が裏付けられるわけではない。

6. 提案する評価計画：未実施

本節の実験は、本稿のためには実施していない。固定したリポジトリは以前の探索的な観測に言及するが、そのベンチマークの生データ、設定から結果までの完全な追跡情報、図の生成一式を提供していない。そのため、本稿の実証結果からは除外する。以下は、手法を検証するために必要な評価計画である。

6.1. 研究課題と統制されたタスク

主要な問いは、SLAが状態だけの構成、および同じメモリ予算のアーカイブだけの手法の両方に対して、未知のテスト系列で再構成誤差を改善するかどうかである。副次的な問いは、どの条件で性能が低下するかである。有利な条件の集約値だけでなく、要因を切り分けた合成タスク群を用いる。

- **独立な対応関係：** 正規化したランダムキーと独立に生成した値を用い、利用できる線形写像のない条件で記憶を調べる。
- **構造と例外を含む値：** $v_i = A^\top k_i + \epsilon_i + z_i r_i$とし、行列、ノイズ、例外率、例外の大きさを公開した生成器で規定する。動機となる仮説を前提とせず検証する。
- **クエリの摂動：** $q = \mathrm{unit}(k_i + \sigma\xi)$を用い、分布と、較正範囲内外の$\sigma$を明示する。
- **新しさとアクセス分布の変化：** 一様、直近重視、反復キーの問い合わせを比較し、較正後にアクセス分布が変わる条件も設ける。
- **衝突とフォールバックの検証：** 矛盾する値を持つ近接キー、異なるIDに対する同一キー、目標エントリがアーカイブに残っていないクエリを含める。

目標が保持中の場合と除去済みの場合を分けて報告する。「アーカイブにない」と「状態が一度も観測していない」は別条件である。同一IDを繰り返し書く場合の正解規約を定め、上書きからの経過と挿入からの経過を別に報告する。未観測クエリについては、誤差の評価前に正解を定める生成写像を規定する。nullへ重みを割り当てた出力を一律に正しい棄権と見なしてはならない。

6.2. ベースラインとメモリ予算の公平性

状態のみのデルタメモリ、状態なしのFIFO KV検索、独立に較正したnull・ゼロ出力を持つFIFO KV、既定のSLA、各λの残差変種を比較する。明示的なフォールバックを持つハードtop-1検索を加え、厳密な設定が得られた段階で、別途コミットを固定したSDA実装も比較する。全履歴KV検索は容量制限なしの参照として利用できるが、同一予算の比較相手ではない。学習済みTransformer、GDN、Kimi Delta Attention（KDA）[10]との比較は別段階の実験であり、未学習のメモリ試験から順位付けしてはならない。

公平性は二つの方法で示す。**構成要素を固定する比較**では、アーカイブ内容、容量、状態、クエリ系列を揃えて読出しの違いを調べる。**予算を固定する比較**では、各手法に同じ総メモリ量を認める。配列本体だけを数える場合、アーカイブのみの手法は同じ型で

$$B_{\mathrm{KV}} = \left\lfloor \frac{d_k d_v + B(d_k + d_v)}{d_k + d_v} \right\rfloor \tag{15}$$

個のエントリを使用できる。メタデータや実行時の一時領域は別途測定する。同じBだけで比較すると、SLAが追加で使う状態の予算を隠してしまう。

6.3. 較正、統計、結果報告

互いに重ならない較正用・テスト用シード、生成器の版、ハイパーパラメータ探索全体を公開する。s, c, α, βとハード検索の閾値は較正データだけで選び、比較手法に同程度の探索予算を与える。$\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$は事前に定義したアブレーションとして評価し、テスト結果を見て最良値を選ぶオラクルにはしない。占有数と容量を変え、式(12)の影響を直接検証する。

各条件について、平均二乗再構成誤差、正規化を明示した誤差、事前定義した許容誤差以内の再現率を報告する。分母と、正解値のエネルギーがほぼゼロの場合の扱いを明示する。アーカイブ保持率、null重みの分布、目標の古さ別の成績も記録する。同じテスト系列に対する対応付きの差を示し、シード単位の信頼区間を求める。初期設計として、独立なテストシードを少なくとも32個用いることを提案する。同一系列のクエリを独立した実験反復として扱ってはならない。マクロ平均だけでなく各条件を公開し、探索的解析と多重比較への対応を区別する。

6.4. 実行系の測定と将来のモデル評価

ウォームアップ後に書込みと読出しを個別測定し、その後に系列全体を測定する。ハードウェア、ソフトウェアの版、型、スレッド設定、次元、容量、占有数、反復測定の分布を報告する。常駐配列、プロセスのピークメモリ、一時確保を区別する。CPU上のNumPyの測定だけでは、GPUの学習効率やデコード効率を裏付けられない。

言語モデルに関する主張を支えられるのは、後続の学習済みモデルを用いた研究である。そのためには、構造、因果的な統合、微分可能性の扱い、パラメータと学習トークンの予算、データセットとライセンス、最適化設定、複数の学習シード、および適切な学習済み比較手法が必要になる。HOLAなどの近いキャッシュ拡張手法を比較に含めるか、省略の理由を説明する必要がある。現状の参照実装は、これらの実験を提供していない。

7. 限界と解釈上の注意

アーカイブの保持範囲は有限であり、除去済みの観測値は別の構成要素が偶然正しく予測しない限り厳密には回復できない。FIFOは再現しやすいが、あらゆる用途に適するわけではない。同一IDの上書きが以前のFIFO位置を保つため、LRUを想定した利用者の期待に反し、直近に更新した項目が早く消えることもある。衝突したキーは、読出し時にIDを使って識別できない。

null候補はフォールバック経路を作るが、意味的な棄権を実現するものではない。有限のsoftmax混合は完全一致を弱めたり、矛盾する値を混ぜたりする。スコア倍率にも残差混合にも、分布に依存しない最適値は示されていない。状態減衰は情報の保持期間を変え、そのアーカイブ保持との相互作用は実証すべき課題である。

実装が扱うのは単一のメモリインスタンスであり、マルチヘッドモデル、バッチ処理、位置表現、学習目的、運用時のサービング方針は定義されていない。単純さは検証に有用だが、他の条件への一般化を制限する。解析結果は意図的に基本的かつ局所的であり、汎化定理、最適な保持方針、先行手法に対する優位性、エンドツーエンドの数値安定性を示すものではない。

明示的なキャッシュは機微な値をそのまま保持し得る。参照実装には、プライバシー保証、過去の状態寄与を消去する手順、安全なメモリのライフサイクル管理がない。運用には用途に応じた保持期間とアクセス制御が必要であり、漸近的なメモリ量の上界だけではこれらの問題を解決できない。

8. 結論

SLAは、デルタ状態の予測器、有界の元KVアーカイブ、nullを含むソフト読出しを組み合わせた、コードと対応するコンパクトな事例である。現段階での価値は、保存、経路選択、上書き、失敗条件を精密に説明できる点にある。解析は、デルタ書込みの局所的な補正作用とソフト検索の限界を示し、1エントリしかない場合でも厳密な想起が失敗し得ることを明らかにした。次に必要な科学的段階は、本稿で規定した、条件を統制しメモリ予算を揃えた評価である。その証拠が得られるまでは、SLAは実験的な参照実装として提示すべきであり、学習済み注意機構に代わることが実証された手法として扱うべきではない。

付録A. 成果物と再現性に関する記載

仕様の根拠はSummerLinearAttentionのコミット036512212ed0である。完全な識別子は`036512212ed0f7d2456988cb4b523599064a7e6f`。実装はreference.pyにあり、パッケージはPython 3.10以上、NumPy 1.24以上を指定する。コードのライセンスはMITである。

リポジトリはインストール、機能確認、実行例を次のように案内している。

```
python -m pip install -e .
python -m unittest discover -s tests -v
python -m examples.quickstart
```

公開されたテストは、容量ゼロ、同一IDの上書き、同じキーを持つ異なるID、FIFO除去、不正な書込み、大きな値の正規化、lambdaゼロの同値性、配列バイト数、単一キーの残差補正を対象とする。ここでテストを列挙したことは、本稿のためにそれらを実行したという主張ではない。投稿前には実行ログと環境情報を保存する必要がある。実証結果を加える前には、生成器、較正・テスト分割、シード別の生出力、解析コード、作図コード、および報告値と成果物の一致を確かめる検査を公開する必要がある。

参考文献

[1] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML, PMLR 119, 5156–5165. Publisher record.

[2] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. 2021. Linear Transformers Are Secretly Fast Weight Programmers. ICML, PMLR 139, 9355–9366. Publisher record.

[3] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2024. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. arXiv:2406.06484.

[4] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated Delta Networks: Improving Mamba2 with Delta Rule. ICLR. Official publication record; arXiv:2412.06464.

[5] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. arXiv:1609.07843.

[6] Wanyun Cui. 2026. A Hippocampus for Linear Attention: An Exact Memory for What the Recurrent State Forgets. arXiv:2607.02303v1.

[7] Mutian He and Philip N. Garner. 2025. Alleviating Forgetfulness of Linear Attention by Hybrid Sparse Attention and Contextualized Learnable Token Eviction. arXiv:2510.20787v2.

[8] summer. 2026. Summer Linear Attention: NumPy Reference Implementation, v0.1.0. Pinned source repository. MIT license. Accessed 24 October 2026.

[9] summer. Summer Delta Attention. Companion project. Repository. Cited for project context, not as empirical evidence for SLA. Accessed 24 October 2026.

[10] Kimi Team. 2025. Kimi Linear: An Expressive, Efficient Attention Architecture. arXiv:2510.26692v2.